From the Transformer to GPT

Document 4 of 6 — Building a GPT from Scratch

The original transformer is a general-purpose sequence-to-sequence architecture. GPT is what happens when you take the decoder half, throw away everything else, and train it to predict the next token. This document traces that path: which pieces survive, what changes, and why those changes matter.

1 Three Transformer Variants

The "Attention Is All You Need" architecture spawned three distinct families, each keeping a different subset of the original design.

1.1 Encoder-Decoder (Vaswani et al., 2017)

This is the full architecture from Document 03. The encoder processes the source sequence with bidirectional self-attention (every token attends to every other token). The decoder generates the target sequence one token at a time with causal self-attention (each token attends only to earlier tokens), plus cross-attention layers that let the decoder read the encoder's output.

Use case: sequence-to-sequence tasks where the input and output are structurally different. Machine translation, summarization, speech-to-text.

1.2 Encoder-Only (BERT, Devlin et al., 2018)

Keep only the encoder stack. Every token attends to every other token in both directions. There is no autoregressive generation — the model processes the entire sequence at once.

Training objective: **masked language modeling (MLM)**. Randomly mask 15% of input tokens, and the model predicts what the masked tokens were. This forces the model to build rich bidirectional representations because every token needs context from both sides to predict the masked positions.

Use case: understanding tasks where you have the full input available — text classification, named entity recognition, extractive question answering. BERT and its descendants (RoBERTa, ALBERT, DeBERTa) dominated NLP benchmarks from 2018 to 2022.

1.3 Decoder-Only (GPT, Radford et al., 2018)

Keep only the decoder stack, and remove the cross-attention sublayer (since there is no encoder to attend to). Every layer is just causal self-attention followed by a feed-forward network — exactly the transformer block from Document 02.

Training objective: **next-token prediction**. Given the sequence so far, predict the next token. The causal mask ensures each position can only see tokens before it.

Use case: generation. But here is the key insight that made decoder-only models dominant: **any task can be framed as "continue this text."**

- Classification: "Review: 'This movie was terrible.' Sentiment:" → the model continues with "negative"
- Translation: "English: Hello. French:" → "Bonjour"
- Question answering: "Context: ... Question: Who won? Answer:" → "The home team"

GPT-2 demonstrated this with zero-shot task performance. GPT-3 (Brown et al., 2020) made it definitive with **in-context learning**: by placing a few examples in the prompt, the model could perform new tasks without any gradient updates. This "few-shot" capability scaled with model size and training data, and it is the reason decoder-only architectures became the foundation for modern large language models.

2 What GPT Removes

Recall from Document 03 that the original decoder layer has three sublayers:

1. Masked multi-head self-attention
2. Multi-head cross-attention (attending to encoder output)
3. Position-wise feed-forward network

GPT has no encoder, so there is nothing to cross-attend to. The cross-attention sublayer is deleted entirely. Each GPT layer has exactly two sublayers:

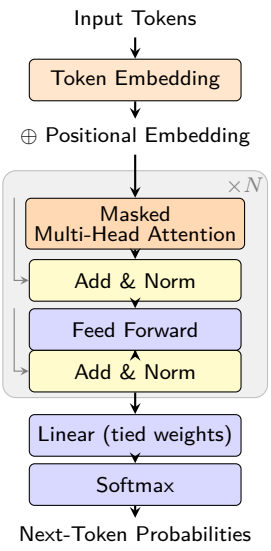
1. Masked multi-head self-attention
2. Position-wise feed-forward network

Here is the side-by-side comparison:

```
Original Decoder Layer (3 sublayers)     GPT Block (2 sublayers)
-------------------------------------    -------------------------
x                                        x
|                                        |
+-- Masked Self-Attention                +-- Masked Self-Attention
|   + Add & Norm                         |   + Add & Norm
|                                        |
+-- Cross-Attention (to encoder)         |  (removed -- no encoder)
|   + Add & Norm                         |
|                                        |
+-- Feed-Forward                         +-- Feed-Forward
|   + Add & Norm                         |   + Add & Norm
|                                        |
output                                   output
```

That is the entire structural difference. The self-attention and feed-forward sublayers work identically to what we built in Document 02.

Compare the decoder-only architecture to the full encoder-decoder from Document 03 — the entire left half and the cross-attention sublayer are gone:



3 What GPT Changes

Beyond removing cross-attention, GPT introduces several modifications that are now standard practice. None of these are architecturally dramatic, but they each solve a specific problem.

3.1 Learned Positional Embeddings

The original paper uses fixed sinusoidal positional encodings — the position signal is computed from a formula and added to the token embeddings. GPT replaces this with a learned positional embedding: a plain `nn.Embedding` table with one vector per position.

Conceptual — illustrating the idea

This line appears inside `BabyGPT.__init__` in `src/model.py`.

```
# Sinusoidal (original paper): computed from a formula, not learned
# Learned (GPT): just another embedding table
self.wpe = nn.Embedding(block_size, n_embd)  # shape: (256, 384)
```

The learned embedding for position `t` is looked up and added to the token embedding, just like the sinusoidal encoding was. The difference is that the model learns the position representations from data rather than having them fixed by a formula.

Why this works: for a fixed maximum sequence length (our `block_size=256`), there is no advantage to the sinusoidal formula. A learned embedding has enough capacity to represent any position pattern the model needs. The sinusoidal encoding was designed to generalize to longer sequences than seen during training (via extrapolation of the sinusoidal functions), but in practice this extrapolation does not work well, and most models are trained and used within a fixed context window.

Tip: `nn.Embedding` vs. `nn.Linear`. An `Embedding(V, d)` is a lookup table: the input is an integer index, the output is the corresponding row of the weight matrix — no multiplication, just `weight[index]`. A `Linear(d_in, d_out)` is a matrix multiply: the input is a continuous vector, the output is `x @ weight.T + bias`. The connection: if you one-hot encoded a token index and multiplied by a weight matrix, you would select exactly one row — the same result as an Embedding lookup, but without materializing a sparse one-hot tensor. This is also why weight tying works: `lm_head` (a Linear) computes `hidden @ wte.weight.T`, which dot-products the hidden state against every embedding row — asking "which embedding vector is this hidden state closest to?"

3.2 GELU Instead of ReLU

The original transformer uses ReLU in the feed-forward network. GPT replaces it with GELU, which is smooth everywhere and does not hard-zero negative inputs. This was covered in detail in Document 02, Section 4. Our `FeedForward` module uses `F.gelu`.

3.3 Pre-Norm Instead of Post-Norm

The original paper applies LayerNorm *after* the residual addition (post-norm). GPT applies it *before* the sublayer (pre-norm), giving the residual connection a clean identity path for gradient flow. This was covered in detail in Document 02, Section 3. Our `TransformerBlock` uses pre-norm: `x = x + sublayer(layer_norm(x))`.

3.4 Weight Tying

The token embedding matrix `wte` has shape `(vocab_size, n_embd)` — it maps token IDs to embedding vectors. The language model head `lm_head` has shape `(n_embd, vocab_size)` — it maps the final hidden state back to logits over the vocabulary. These two matrices are transposes of each other conceptually: one maps from token space to embedding space, the other maps back.

Weight tying forces them to share the same weight matrix:

Conceptual — illustrating the idea

From `BabyGPT.__init__` in `src/model.py`.

```
# In BabyGPT.__init__:
self.transformer.wte.weight = self.lm_head.weight
```

This means `lm_head` computes logits as a dot product between the hidden state and the token embeddings. A token's logit is high when the hidden state is close to that token's embedding vector. This reduces the parameter count (one less `vocab_size x n_embd` matrix) and acts as regularization — the model is encouraged to produce embeddings that are directly useful for prediction.

Tip: If `lm_head` shares weights with `wte`, how does it predict the *next* token? The hidden state at position t *starts* as the embedding of x_t, but 6 transformer blocks then transform it — attention mixes in context from positions $0 \ldots t$, and feed-forward layers apply nonlinear transformations. By the time it exits the final LayerNorm, it is no longer "the embedding of x_t." The loss function forces the transformer blocks to push that hidden vector **toward the embedding of** x_{t+1}, because the tied head computes $\text{logit}[i] = \text{dot}(\text{hidden}_t,\ \text{embedding}_i)$, and the loss rewards maximizing the logit for the correct next token. Think of the embedding space as a shared room where every character has a fixed address: the transformer blocks move the hidden state from near x_t's address toward x_{t+1}'s address, and the tied head asks "whose address are you closest to now?" Document 05 covers the loss function in detail.

3.5 Dropout

Dropout (Srivastava et al., 2014) is a regularization technique. During training, each activation is independently zeroed out with probability p:

$$\text{Dropout}(x)_i = \begin{cases} 0 & \text{with probability } p \\ x_i \,/\, (1-p) & \text{with probability } 1-p \end{cases}$$

The scaling by $1/(1-p)$ keeps the expected value of each element unchanged, so downstream layers see the same magnitude whether dropout is on or off.

Why this works: dropout prevents **co-adaptation** — neurons cannot rely on specific other neurons always being present, so the network learns more robust, redundant representations. Each forward pass trains a different random sub-network (some neurons removed, others kept), and they all share the same weights. The full network at inference time acts like an implicit ensemble of all those sub-networks.

During inference (`model.eval()`), dropout is disabled entirely — all activations pass through at full strength.

Our model uses $p = 0.2$ and applies dropout in three places:

1. **After the embedding addition** (`transformer.drop`): regularizes the combined token + position signal before it enters the transformer blocks.
2. **After attention weights** (`attn_dropout` in `CausalSelfAttention`): randomly drops connections between positions, forcing the model not to over-rely on any single attention pattern.
3. **After each sublayer's output projection** (`resid_dropout` in attention, `dropout` in FFN): regularizes the signal before it is added back via the residual connection.

The original transformer paper applies dropout in the same three locations with $p = 0.1$. We use a higher rate because our model is small and our dataset is small — stronger regularization helps prevent overfitting.

4 The Language Modeling Objective

GPT is trained with a single objective: predict the next token.

Given a sequence of tokens $x_1, x_2, \ldots, x_T$, the model learns to maximize the probability of each token given all preceding tokens. The loss function is the average negative log-likelihood:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_t \mid x_1, x_2, \ldots, x_{t-1})$$

In plain English: for every position in the sequence, the model outputs a probability distribution over the vocabulary. The loss measures how much probability the model assigned to the token that actually came next, averaged over all positions. Lower loss means the model is better at predicting the next token.

This is computed as cross-entropy loss at every position simultaneously. The causal mask ensures that position t can only attend to positions $1, \ldots, t-1$, so each position's prediction genuinely uses only past context — even though all positions are processed in one forward pass.

This is **teacher forcing**: during training, each position sees the ground-truth tokens before it, not the model's own predictions. This allows the entire sequence to be processed in parallel (one forward pass), which is far more efficient than generating one token at a time.

Note how this differs from the encoder-decoder objective. In machine translation, you predict target tokens given a separate source sequence. In language modeling, the source and target are the same sequence, shifted by one position.

5 Our BabyGPT Architecture

The full model is in `src/model.py`. Here is the architecture with our specific configuration from `src/config.py`:

Parameter	Value	Meaning
`block_size`	256	Maximum sequence length (context window)
`n_layer`	6	Number of transformer blocks
`n_head`	6	Number of attention heads per block
`n_embd`	384	Embedding dimension (d_{model})
`dropout`	0.2	Dropout rate
`vocab_size`	65	Number of unique characters
`bias`	False	No bias terms in Linear/LayerNorm

5.1 Components

The model is organized as a `ModuleDict` called `transformer` plus a separate `lm_head`:

- **`wte`**: Token embedding — `nn.Embedding(65, 384)`. Maps each of the 65 character IDs to a 384-dimensional vector.
- **`wpe`**: Position embedding — `nn.Embedding(256, 384)`. Maps each position index (0 through 255) to a 384-dimensional vector.
- **`drop`**: Dropout applied after embedding addition.
- **`blocks`**: A `ModuleList` of 6 `TransformerBlock` layers. Each block contains a `CausalSelfAttention` and a `FeedForward` sublayer, each preceded by `LayerNorm` and followed by a residual connection.
- **`ln_f`**: A final `LayerNorm` applied after the last block (required for pre-norm architectures, since the residual path bypasses the per-block norms).
- **`lm_head`**: `nn.Linear(384, 65, bias=False)`. Projects the final hidden states to logits over the vocabulary. Weight-tied with `wte`.

Tip: Why `c_fc` and `c_proj`? These names come from OpenAI's GPT-2 codebase, which used 1D convolutions (`Conv1D`) instead of `nn.Linear` for these layers — a 1×1 convolution over the sequence dimension is mathematically identical to a per-position linear projection. The `c_` prefix stands for "convolution"; `fc` = fully connected (the expand layer), `proj` = projection (the contract/output layer). We keep the names so our weight dictionaries match GPT-2 checkpoints, which is useful if you ever want to load pretrained GPT-2 weights. The layers themselves are standard `nn.Linear`.

5.2 Forward Pass

Conceptual — illustrating the idea

From `BabyGPT.forward` in `src/model.py`.

```
def forward(self, idx: torch.Tensor, targets: torch.Tensor | None =
    None):
    B, T = idx.shape
    pos = torch.arange(T, device=idx.device)

    tok_emb = self.transformer.wte(idx)      # (B, T, 384)
    pos_emb = self.transformer.wpe(pos)      # (T, 384) -- broadcasts
        over batch
    x = self.transformer.drop(tok_emb + pos_emb)

    for block in self.transformer.blocks:
        x = block(x)                          # 6 transformer blocks

    x = self.transformer.ln_f(x)             # final layer norm
    logits = self.lm_head(x)                 # (B, T, 65)

    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1),
        )
    return logits, loss
```

The input `idx` has shape `(B, T)` — a batch of B sequences, each T tokens long. The output `logits` has shape `(B, T, 65)` — a probability distribution over 65 characters at every position. When `targets` are provided, cross-entropy loss is computed and returned alongside the logits.

Tip: Why does each position predict the *next* token? The causal mask ensures position t can only attend to tokens $x_0, \ldots, x_t$ — it cannot see x_{t+1} or beyond. So the hidden vector at position t is a learned summary of the past. The loss function (covered in Document 05) compares position t's output against the target x_{t+1}. Gradients flow backward and reshape the model so that each position's representation becomes useful for predicting what comes next — because that is the only thing the loss ever rewards. The architecture does not force this; training does.

6 Parameter Count

Let us count every trainable parameter in the model.

Token embedding (`wte`):

$$65 \times 384 = 24{,}960$$

Position embedding (`wpe`):

$$256 \times 384 = 98{,}304$$

Per transformer block:

The attention sublayer has three Q, K, V projections and one output projection:

- `W_Q`, `W_K`, `W_V`: $3 \times (384 \times 384) = 3 \times 147{,}456 = 442{,}368$
- `c_proj` (output): $384 \times 384 = 147{,}456$
- Total attention weights: $442{,}368 + 147{,}456 = 589{,}824$

Tip: How multi-head attention slices the matrices. Each `W_Q`, `W_K`, `W_V` projection is a single $(384, 384)$ matrix, but conceptually it is $h = 6$ separate $(384, 64)$ projections stacked side by side. The per-head dimension is $d_k = n_{\text{embd}}/h = 384/6 = 64$. In code, we project the full 384-dim vector first, then `view` the result as $(B, T, 6, 64)$ and `transpose` to $(B, 6, T, 64)$ — giving each head its own 64-dim Q, K, V to run attention on independently. After attention, the 6 heads of 64 dims are concatenated back to 384 and passed through `c_proj`. One big matrix multiply is equivalent to 6 small ones, but runs faster on a GPU because it is a single contiguous operation.

The feed-forward sublayer has two linear layers with a 4x expansion:

- `c_fc` (expand): $384 \times 1536 = 589{,}824$
- `c_proj` (contract): $1536 \times 384 = 589{,}824$
- Total FFN weights: $589{,}824 + 589{,}824 = 1{,}179{,}648$

Two LayerNorm layers (weight only, no bias since `bias=False`):

- $2 \times 384 = 768$

Total per block: $589{,}824 + 1{,}179{,}648 + 768 = 1{,}770{,}240$

All 6 blocks:

$$6 \times 1{,}770{,}240 = 10{,}621{,}440$$

Final LayerNorm (`ln_f`):

$$384$$

Language model head (`lm_head`): Weight-tied with `wte`, so 0 additional parameters.

Total:

$$24{,}960 + 98{,}304 + 10{,}621{,}440 + 384 = 10{,}745{,}088 \approx 10.7\text{M}$$

This is a small model by modern standards (GPT-3 is 175B parameters), but large enough to learn interesting character-level patterns from TinyShakespeare.

7 Parameter Initialization

Random initialization matters more than you might expect. If initial weights are too large, activations explode and training diverges. If too small, gradients vanish and the model learns nothing.

Standard weights: All weight matrices and embeddings are initialized from a normal distribution with mean 0 and standard deviation 0.02:

Conceptual — illustrating the idea
From `BabyGPT.__init__` and `BabyGPT._init_weights` in `src/model.py`.

```
nn.init.normal_(module.weight, mean=0.0, std=0.02)
```

Biases: Initialized to zero (though our model has `bias=False`, so this only applies if you enable biases):

Conceptual — illustrating the idea
From `BabyGPT.__init__` and `BabyGPT._init_weights` in `src/model.py`.

```
nn.init.zeros_(module.bias)
```

Residual projection scaling: The output projection (`c_proj`) in both the attention and feed-forward sublayers receives special treatment. These are the projections whose outputs get added back via the residual connection.

Conceptual — illustrating the idea
From `BabyGPT.__init__` and `BabyGPT._init_weights` in `src/model.py`.

```
for pn, p in self.named_parameters():
    if pn.endswith("c_proj.weight"):
        nn.init.normal_(p, mean=0.0, std=0.02 / math.sqrt(2 *
            config.n_layer))
```

The scaling factor is $1/\sqrt{2N}$ where N is the number of transformer blocks.

Why this is necessary: consider what happens at initialization. Each transformer block adds the output of its two sublayers (attention and FFN) to the residual stream. If each sublayer's output has variance σ^2, then after N blocks with 2 residual additions each, the residual stream's variance has grown by approximately $2N \cdot \sigma^2$ (assuming the additions are roughly independent).

Scaling the residual branch outputs by $1/\sqrt{2N}$ reduces their variance by a factor of $2N$, so the total variance contribution from all residual additions is approximately $2N \cdot \sigma^2/(2N) = \sigma^2$. The residual stream's variance stays bounded regardless of depth.

For our model with $N = 6$: the scaling factor is $0.02/\sqrt{12} \approx 0.00577$. This is the GPT-2 initialization convention.

8 Building the Model

We now have everything we need to write the model code. Create two files:

8.1 Step 1: Configuration

File: src/config.py

```python
from dataclasses import dataclass

@dataclass
class GPTConfig:
    block_size: int = 256
    vocab_size: int = 65
    n_layer: int = 6
    n_head: int = 6
    n_embd: int = 384
    dropout: float = 0.2
    bias: bool = False
```

8.2 Step 2: The Model

Create `src/model.py` with the complete implementation. Every class from the previous documents is collected here.

File: src/model.py

```python
"""BabyGPT: a decoder-only transformer language model."""

import math

import torch
import torch.nn as nn
import torch.nn.functional as F

from src.config import GPTConfig


class LayerNorm(nn.Module):
    """Hand-written LayerNorm with optional bias (Document 02, Section
        3).

    LayerNorm(x) = gamma * (x - mean) / sqrt(var + eps) + beta
    Normalizes across the last dimension (features) per position.
    gamma (weight) and beta (bias) are learnable affine parameters.
    """

    def __init__(self, config, eps=1e-5):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(config.n_embd))     # gamma
        self.bias = nn.Parameter(torch.zeros(config.n_embd)) if
            config.bias else None  # beta
        self.eps = eps

    def forward(self, x):
        mu = x.mean(dim=-1, keepdim=True)
        sigma_sq = x.var(dim=-1, keepdim=True, unbiased=False)
        x_normalized = (x - mu) / torch.sqrt(sigma_sq + self.eps)
        out = self.weight * x_normalized
        if self.bias is not None:
            out = out + self.bias
        return out
```

```python
class CausalSelfAttention(nn.Module):
    """Multi-head self-attention with causal mask (Document 01).

    Attention(Q, K, V) = softmax(Q K^T / sqrt(d_k)) V
    Runs h heads in parallel by reshaping into (B, h, T, d_k).
    Causal mask sets future positions to -inf before softmax.
    """

    def __init__(self, config: GPTConfig):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        # Each projection is (n_embd, n_embd), i.e. (384, 384).
        # Conceptually this is h separate (n_embd, head_dim) projections
        # stacked together: 6 heads x 64 dims = 384.
        self.W_Q = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.W_K = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.W_V = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd,
            bias=config.bias)
        self.attn_dropout = nn.Dropout(config.dropout)
        self.resid_dropout = nn.Dropout(config.dropout)
        self.n_head = config.n_head       # h = 6
        self.n_embd = config.n_embd       # 384
        self.head_dim = config.n_embd // config.n_head  # d_k = 384/6 =
            64

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        B, T, C = x.shape  # C = n_embd = 384

        # Project then split into heads:
        # (B, T, 384) -> (B, T, 384) -> view as (B, T, 6, 64) -> (B, 6,
            T, 64)
        # Each head gets its own 64-dim slice of Q, K, V.
        q = self.W_Q(x).view(B, T, self.n_head,
            self.head_dim).transpose(1, 2)
        k = self.W_K(x).view(B, T, self.n_head,
            self.head_dim).transpose(1, 2)
        v = self.W_V(x).view(B, T, self.n_head,
            self.head_dim).transpose(1, 2)

        # Attention per head: (B, 6, T, 64) @ (B, 6, 64, T) -> (B, 6, T,
            T)
        scores = torch.matmul(q, k.transpose(-2, -1)) /
            math.sqrt(self.head_dim)
        mask = torch.tril(torch.ones(T, T, device=x.device,
            dtype=torch.bool))
        scores = scores.masked_fill(~mask, float('-inf'))
        weights = F.softmax(scores, dim=-1)
        weights = self.attn_dropout(weights)
        y = torch.matmul(weights, v)  # (B, 6, T, 64)

        # Reassemble heads: (B, 6, T, 64) -> (B, T, 6, 64) -> (B, T, 384)
        y = y.transpose(1, 2).contiguous().view(B, T, C)
```

```python
        return self.resid_dropout(self.c_proj(y))


class FeedForward(nn.Module):
    """Position-wise FFN with GELU (Document 02, Section 4).

    FFN(x) = W2 * GELU(W1 * x)
    Hidden dim is 4x the embedding dim (expand then contract).
    """

    def __init__(self, config: GPTConfig):
        super().__init__()
        self.c_fc = nn.Linear(config.n_embd, 4 * config.n_embd,
            bias=config.bias)
        self.c_proj = nn.Linear(4 * config.n_embd, config.n_embd,
            bias=config.bias)
        self.dropout = nn.Dropout(config.dropout)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.c_fc(x)
        x = F.gelu(x)
        x = self.c_proj(x)
        x = self.dropout(x)
        return x


class TransformerBlock(nn.Module):
    """Pre-norm transformer block (Document 02, Section 5).

    x = x + Attention(LayerNorm(x))    -- pre-norm + residual
    x = x + FFN(LayerNorm(x))          -- pre-norm + residual
    """

    def __init__(self, config: GPTConfig):
        super().__init__()
        self.ln_1 = LayerNorm(config)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = LayerNorm(config)
        self.ffn = FeedForward(config)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = x + self.attn(self.ln_1(x))
        x = x + self.ffn(self.ln_2(x))
        return x


class BabyGPT(nn.Module):
    """Decoder-only transformer language model (this document).

    Forward: tok_emb + pos_emb -> N x TransformerBlock -> LayerNorm ->
        lm_head
    Loss: cross-entropy over vocab at every position (next-token
        prediction).
    Weight tying: lm_head shares weights with token embedding.
    """

    def __init__(self, config: GPTConfig):
        super().__init__()
```

```python
        self.config = config

        self.transformer = nn.ModuleDict(dict(
            wte=nn.Embedding(config.vocab_size, config.n_embd),
            wpe=nn.Embedding(config.block_size, config.n_embd),
            drop=nn.Dropout(config.dropout),
            blocks=nn.ModuleList(
                [TransformerBlock(config) for _ in range(config.n_layer)]
            ),
            ln_f=LayerNorm(config),
        ))
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size,
           bias=False)

        # Weight tying
        self.transformer.wte.weight = self.lm_head.weight

        # Initialize weights
        self.apply(self._init_weights)
        # Scaled init for residual projections (GPT-2 convention)
        for pn, p in self.named_parameters():
            if pn.endswith("c_proj.weight"):
                nn.init.normal_(
                    p, mean=0.0, std=0.02 / math.sqrt(2 * config.n_layer)
                )

    def _init_weights(self, module: nn.Module) -> None:
        if isinstance(module, nn.Linear):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx: torch.Tensor, targets: torch.Tensor | None =
       None):
        B, T = idx.shape
        assert T <= self.config.block_size
        pos = torch.arange(T, device=idx.device)

        tok_emb = self.transformer.wte(idx)       # (B, T, n_embd)
        pos_emb = self.transformer.wpe(pos)       # (T, n_embd)
        x = self.transformer.drop(tok_emb + pos_emb)

        for block in self.transformer.blocks:
            x = block(x)

        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)                  # (B, T, vocab_size)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(
                logits.view(-1, logits.size(-1)),
                targets.view(-1),
            )
        return logits, loss
```

```python
    def count_parameters(self) -> int:
        """Trainable parameters (excluding tied duplicates)."""
        n = sum(p.numel() for p in self.parameters())
        n -= self.lm_head.weight.numel()  # tied with wte
        return n
```

8.3 Step 3: Verify

Shell — run in your terminal

```shell
uv run python -c "
from src.config import GPTConfig
from src.model import BabyGPT
import torch
model = BabyGPT(GPTConfig())
print(f'Parameters: {model.count_parameters():,}')
x = torch.randint(0, 65, (2, 256))
logits, loss = model(x, torch.randint(0, 65, (2, 256)))
print(f'Logits: {logits.shape}, Loss: {loss.item():.2f}')
"
```

Checkpoint: You should see `Parameters: 10,720,128` and a loss near 4.17 (which is $\ln(65)$, the random baseline for 65-class classification).